

Model_and_Stream

October 17, 2025

1 Predicting on Streaming Data using Machine Learning

To showcase my grasp of machine learning with PySpark, I'll walk through a couple of key scenarios. First, under the umbrella of ***Fitting Your Model***, I'll demonstrate the process of reading in a static dataset (our 'batch'), performing initial summarization, transforming it into a suitable format, and constructing a machine learning pipeline for training and evaluation. Then, shifting gears to ***Modeling on Streaming***, I'll tackle a similar dataset but ingest it incrementally. As new data arrives in the Jupyter environment, I'll show how to update predictions in near real-time.

1.0.1 Fitting Your Model

Setting up the Environemnt, reading in the data, conversion to workable formatting. The initial step involves importing the necessary Python modules for the subsequent tasks. Consistent with practices often employed in authoring RMarkdown documents, I have chosen to consolidate all module loading in a dedicated section at the beginning of this notebook. This centralized arrangement facilitates efficient management and debugging, should any issues arise during processing. As the analysis progresses, any additional required modules will be imported into this designated section as needed.

```
[1]: # seaborn install for a heatmap used in the summary section.
!pip install seaborn

# Code Chunk handles loading the modules required for this assignment ordered by
↳ top-level packages, sub-packages, and submodules.
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from pyspark import SparkContext
from pyspark.sql import SparkSession, Row
from pyspark.sql.types import DoubleType, StringType, StructType, StructField,
↳ LongType
from pyspark.sql.functions import col, round, count, avg, stddev, when
from pyspark.ml import Pipeline
from pyspark.ml.feature import Binarizer, OneHotEncoder, StringIndexer, PCA,
↳ VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
```

```
from pyspark.ml.evaluation import RegressionEvaluator
```

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in /home/jupyter-blakerma@ncsu.edu/.local/lib/python3.9/site-packages (0.13.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/tljh/user/lib/python3.9/site-packages (from seaborn) (3.6.3)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/tljh/user/lib/python3.9/site-packages (from seaborn) (1.24.1)
Requirement already satisfied: pandas>=1.2 in /opt/tljh/user/lib/python3.9/site-packages (from seaborn) (2.2.2)
Requirement already satisfied: fonttools>=4.22.0 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.38.0)
Requirement already satisfied: pyparsing>=2.2.1 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.0.9)
Requirement already satisfied: pillow>=6.2.0 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (9.4.0)
Requirement already satisfied: cycler>=0.10 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.0.7)
Requirement already satisfied: packaging>=20.0 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (22.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.8.2)
Requirement already satisfied: kiwisolver>=1.0.1 in /opt/tljh/user/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: tzdata>=2022.7 in /opt/tljh/user/lib/python3.9/site-packages (from pandas>=1.2->seaborn) (2025.1)
Requirement already satisfied: pytz>=2020.1 in /opt/tljh/user/lib/python3.9/site-packages (from pandas>=1.2->seaborn) (2022.7)
Requirement already satisfied: six>=1.5 in /opt/tljh/user/lib/python3.9/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)

Now I can read-in the power_ml_data.csv file from a URL using pandas. A snapshot of the pandas dataframe is made available just to see what we are working with.

```
[2]: # Code Chunk handles the "reading in" of the ML data into the ipynb environment
ml_df = pd.read_csv('https://www4.stat.ncsu.edu/~online/datasets/power_ml_data.
↳CSV')
```

```
# Brief sample of how the data appears after being read in raw.
ml_df.head(3)
```

```
[2]:      Temperature  Humidity  Wind_Speed  General_Diffuse_Flows  Diffuse_Flows  \
0          6.559      73.8        0.083          0.051          0.119
1          6.414      74.5        0.083          0.070          0.085
2          6.313      74.5        0.080          0.062          0.100

      Power_Zone_1  Power_Zone_2  Power_Zone_3  Month  Hour
0    34055.69620    16128.87538    20240.96386      1     0
1    29814.68354    19375.07599    20131.08434      1     0
2    29128.10127    19006.68693    19668.43373      1     0
```

Next, conversion of the pandas dataframe to a spark dataframe. First, we have to initialize a spark session to use a spark dataframe. The `appName()` levels the spark application for downstream tracking of jobs; meanwhile, the `getOrCreate()` does that—retrieves it if it exists and creates a new one if it does not. `createDataFrame`—self-explanatory. The `createOrReplaceTempView()` is called so that it takes the `spark_df` object and gives it a `TempView` which is then needed so we can run SQL queries against it.

```
[3]: # Begins a Spark session.
spark = SparkSession.builder.appName("PandasToPySpark").getOrCreate()

# I was receiving some warnings when running a LinearRegression() method so
↳ this part silences those warnings.
# REF: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.
↳ SparkContext.setLogLevel.html
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# Conversion to Spark DataFrame
spark_df = spark.createDataFrame(ml_df)

# Register the PySpark DataFrame as a temporary SQL table
# REF: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/
↳ api/pyspark.sql.DataFrame.createOrReplaceTempView.html
spark_df.createOrReplaceTempView("ml_table")

# Brief sample of how the spark dataframe appears after conversion.
spark_df.show(3)
```

Setting default log level to "WARN".

To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.

```
25/04/21 17:01:42 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
25/04/21 17:01:42 WARN Utils: Service 'SparkUI' could not bind on port 4040.
```

Attempting port 4041.
 25/04/21 17:01:42 WARN Utils: Service 'SparkUI' could not bind on port 4041.
 Attempting port 4042.
 25/04/21 17:01:42 WARN Utils: Service 'SparkUI' could not bind on port 4042.
 Attempting port 4043.

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows|Power_Zone_1|Power_Zone_2|Power_Zone_3|Month|Hour|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      6.559|    73.8|    0.083|                0.051|    0.119|
34055.6962| 16128.87538| 20240.96386|    1|    0|
|      6.414|    74.5|    0.083|                0.07|    0.085|
29814.68354| 19375.07599| 20131.08434|    1|    0|
|      6.313|    74.5|    0.08|                0.062|    0.1|
29128.10127| 19006.68693| 19668.43373|    1|    0|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 3 rows
```

Summarize the Data With the data ready to use in this notebook the next move is to look at the summaries of this data. As is typically expected in an exploratory data analysis, we would want a high level view of how our data looks, how it is shaped, how it varies and how the data interacts among its features.

```
[4]: # First, computing the median since it stands alone from the summary() PySpark
      ↪method used to find the rest of the summary values.
      # I will declare the columns I want to take the median for, then compute the
      ↪median for each. Note that Month/Hour are going to
      # hold NaN values because it does not make sense to take the median of
      ↪time-related values.
      numeric_cols = ["Temperature", "Humidity", "Wind_Speed",
      ↪"General_Diffuse_Flows",
      ↪"Diffuse_Flows", "Power_Zone_1", "Power_Zone_2", "Power_Zone_3"]
      median_values = {col_name: float("{:.2f}".format(spark_df.
      ↪approxQuantile(col_name, [0.5], 0.01)[0])) for col_name in numeric_cols}

      # Take the median values and objectify them to a row that I can add to a table
      ↪of summary values I am about to create
      median_row = Row("median", *[str(median_values.get(c, "NA")) for c in spark_df.
      ↪columns])
```

```
# Now create the summary table containing the remainder summary stats (means,
↳ standard deviations, min, max). I am rounding all values to two places for
↳ readability.
summary = spark_df.summary().select(
    "summary",
    *[round(col(c), 2).alias(c) for c in spark_df.columns if c != "summary"]
)

# Append median to table.
summary = summary.union(spark.createDataFrame([median_row]))

# Show the summary table
summary.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|summary|Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows|Pow
er_Zone_1|Power_Zone_2|Power_Zone_3|Month   |Hour   |
+-----+-----+-----+-----+-----+-----+-----+
|count   |47174.0     |47174.0 |47174.0   |47174.0           |47174.0           |
|47174.0  |47174.0     |47174.0 |47174.0   |47174.0|47174.0|
|mean    |18.81       |68.29   |1.96      |182.53            |74.99             |
|32335.17 |21027.2     |17831.2 |6.51      |11.49            |                  |
|stddev  |5.81        |15.56   |2.35      |264.43            |124.26            |
|7130.01  |5199.79     |6622.59 |3.44      |6.92            |                  |
|min     |3.25        |11.34   |0.05      |0.0               |0.01              |
|13895.7  |8560.08     |5935.17 |1.0        |0.0              |                  |
|25%     |14.42       |58.32   |0.08      |0.06              |0.12              |
|26290.63 |16957.32    |13121.93|4.0        |5.0              |                  |
|50%     |18.78       |69.89   |0.09      |4.78              |4.28              |
|32261.6  |20804.86    |16405.28|7.0        |11.0             |                  |
|75%     |22.91       |81.5    |4.92      |319.0             |101.0             |
|37317.45 |24698.73    |21628.92|9.0        |17.0             |                  |
|max     |40.01       |94.8    |6.48      |1163.0            |936.0             |
|52146.86 |37408.86    |47598.33|12.0       |23.0             |                  |
|median  |18.72       |69.67   |0.09      |3.24              |2.87              |
|32157.96 |20722.78    |16349.09|NA         |NA               |                  |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
```

We would want to examine a correlation matrix to see if there exists a relationship between the variables we are working with. The code chunk below produces that output. From what can be seen, there are a few observations to note:

- There is high correlation between power zones—likely the result of all three zones being

powered by the same source.

- Temperature is positively correlated with power demand
- Humidity is negatively correlated with power. I would think that humidity generally means cloudiness, possible cooler days resulting in lower energy output (if solar) or lower demand (less cooling needs)
- There is some relationship between Power_Zone_3 and Wind_Speed. Possibility of some wind farm or geographical feature causing this. We would need more information to explain this relationship.

```
[5]: #Find the correlations between all variables where that makes sense to do
# List of numeric columns
numeric_cols = ["Temperature", "Humidity", "Wind_Speed",
↳"General_Diffuse_Flows",
               "Diffuse_Flows", "Power_Zone_1", "Power_Zone_2", "Power_Zone_3"]

# Compute correlations pairwise
# REF: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/
↳api/pyspark.sql.DataFrame.corr.html
correlation_data = []
for col1 in numeric_cols:
    row_data = [col1] # First element: column name
    for col2 in numeric_cols:
        correlation_value = spark_df.stat.corr(col1, col2)
        row_data.append(float(f"{correlation_value:.2f}")) # Proper rounding
    correlation_data.append(Row(*row_data)) # Convert to Row object

# Create DataFrame from computed correlation values
correlation_df = spark.createDataFrame(correlation_data, ["Variable"] +
↳numeric_cols)

# Show correlation matrix
correlation_df.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|Variable|Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Dif
fuse_Flows|Power_Zone_1|Power_Zone_2|Power_Zone_3|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|Temperature|1.0| -0.46| 0.48| 0.46| |0.2
|0.44| 0.38| 0.49| | | |
|Humidity| -0.46| 1.0| -0.14| -0.47| |
| -0.26| -0.29| -0.3| | -0.23| |
|Wind_Speed| 0.48| -0.14| 1.0| 0.13| |
| -0.0| 0.17| 0.15| | 0.28| |
|General_Diffuse_Flows|0.46| -0.47| 0.13| 1.0| |
|0.56| 0.19| 0.16| | 0.06| |
```

Diffuse_Flows	0.2	-0.26	-0.0	0.56	1.0
0.08	0.05	-0.04			
Power_Zone_1	0.44	-0.29	0.17	0.19	
0.08	1.0	0.83	0.75		
Power_Zone_2	0.38	-0.3	0.15	0.16	
0.05	0.83	1.0	0.57		
Power_Zone_3	0.49	-0.23	0.28	0.06	
-0.04	0.75	0.57	1.0		
+-----+-----+-----+-----+-----+					
-----+-----+-----+-----+					

As part of our assessment on the data, we might be interested in the frequency of observations in our data. **Month** lowest count occur in month two and the highest in month three with a range of 469. **Hour** lowest count is observed in hour fifteen and highest in hour six with a range of forty-five. I do not see anything out of the ordinary here but we would be looking for outliers of observations. From what I can tell, the data frequencies for **Month** and **Hour** are acceptable.

```
[6]: # Contingency table for Month
      month_ct = spark_df.groupBy("Month").agg(count("*").alias("Count")).
        ↪orderBy("Month")
      month_ct.show()

      # Contingency table for Hour
      hour_ct = spark_df.groupBy("Hour").agg(count("*").alias("Count")).
        ↪orderBy("Hour")
      hour_ct.show()
```

Month	Count
1	4014
2	3588
3	4057
4	3893
5	3997
6	3913
7	4029
8	3999
9	3913
10	4026
11	3877
12	3868

+	-	-	-	+	-	-	-	-	+
	Hour		Count						

```

+-----+-----+
|  0| 1950|
|  1| 1973|
|  2| 1973|
|  3| 1966|
|  4| 1986|
|  5| 1968|
|  6| 1992|
|  7| 1964|
|  8| 1957|
|  9| 1976|
| 10| 1955|
| 11| 1972|
| 12| 1979|
| 13| 1956|
| 14| 1971|
| 15| 1947|
| 16| 1950|
| 17| 1979|
| 18| 1955|
| 19| 1950|
+-----+-----+

```

only showing top 20 rows

We might want to look at our data frequencies by Month and by Hour next. This would show us areas in our data that have more representation than others.

```

[7]: # Compute the contingency table for Month and Hour
contingency_table = spark_df.groupBy("Month", "Hour").agg(count("*").
    ↪alias("Count")).orderBy("Month", "Hour")

# Show the contingency table
contingency_table.show(truncate=False)

```

```

+-----+-----+-----+
|Month|Hour|Count|
+-----+-----+-----+
|1     |0    |161  |
|1     |1    |165  |
|1     |2    |168  |
|1     |3    |162  |
|1     |4    |167  |
|1     |5    |168  |
|1     |6    |168  |
|1     |7    |168  |
|1     |8    |172  |
|1     |9    |174  |

```


1	10	164	
1	11	174	
1	12	170	
1	13	165	
1	14	167	
1	15	163	
1	16	169	
1	17	170	
1	18	168	
1	19	162	

+-----+-----+-----+

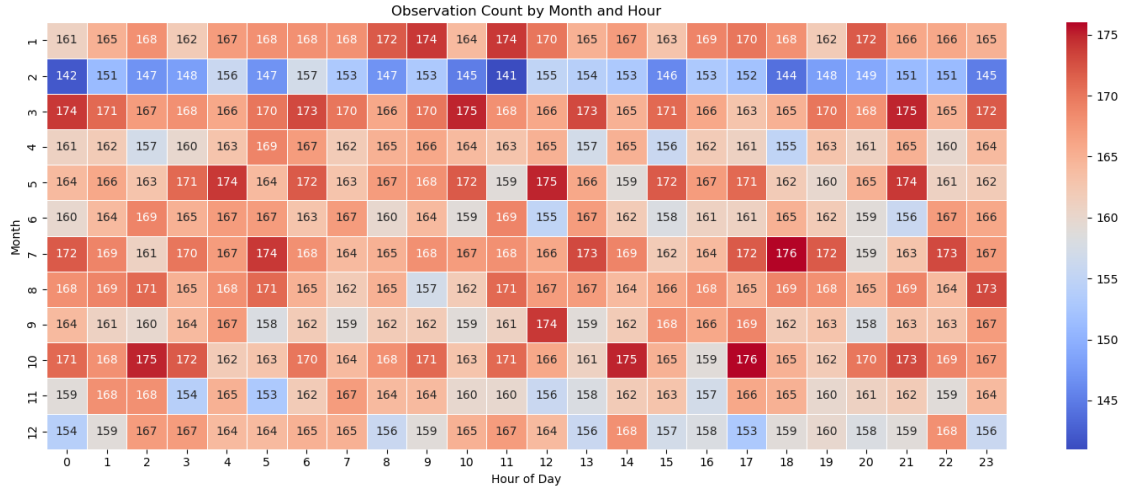
only showing top 20 rows

The two-way is somewhat lengthy and difficult to discern areas of over/under representation. This is a heatmap of the two-way contingency table to help illustrate it better. It shows observations with more data (red) and few data (blue) than usual. The darker reds are scattered about with no patterns which is fine. If you stare at this heatmap long enough, you might see that there are fewer observations in two regions; months 9-12 with hours 5-10, and a second region in months 1-5 with hours 14-20. The **Month** two definitely shows a pattern of fewer observations across the board. I do not know if that is worth looking into or not but I would likely note this observation in any final report.

```
[8]: # Convert the PySpark contingency table back to Pandas to use with seaborn.
# REF: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/
#       ↪ api/pyspark.sql.DataFrame.toPandas.html
contingency_pd = contingency_table.toPandas()

# Pivot it into a matrix for a heatmap
# REF: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/
#       ↪ api/pyspark.sql.GroupedData.pivot.html
heatmap_data = contingency_pd.pivot(index="Month", columns="Hour",
#       ↪ values="Count")

# Plot the Heatmap
# REF: https://seaborn.pydata.org/generated/seaborn.heatmap.html
plt.figure(figsize=(15, 6))
sns.heatmap(heatmap_data, annot=True, fmt="d", cmap="coolwarm", cbar=True,
#       ↪ linewidths=0.5)
plt.title("Observation Count by Month and Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Month")
plt.tight_layout()
plt.show()
```



Gathering summaries by month will help us understand seasonal average. This next code block groups the data by month and obtains the averages for each of the numeric variables. There are some seasonality elements we can observe. Months five through nine have higher mean temperatures, higher mean wind speeds, and higher general diffuse flows on average. All mean power zones show increased output during this time too, on average. Humidity stays fairly consistent on average with the exception of month seven. `Mean_diffuse_flows` does not follow the `general_diffuse_flows` pattern; showing spikes only during months five and six.

```
[9]: # Group the data by Month and find the means of the numeric variables
# Compute mean values for each numeric column grouped by Month
mean_by_month = spark_df.groupBy("Month").agg(
    *[avg(col_name).alias(f"Mean_{col_name}") for col_name in ["Temperature",
    ↪ "Humidity", "Wind_Speed",
    ↪ "General_Diffuse_Flows", "Diffuse_Flows",
    ↪ "Power_Zone_1",
    ↪ "Power_Zone_2", "Power_Zone_3"]])

# Round all numeric columns to 3 decimal places
rounded_means = mean_by_month.select(
    "Month", # Keep Month column unchanged
    *[round(col(c), 3).alias(c) for c in mean_by_month.columns if c != "Month"])

# Show the rounded table
rounded_means.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|Month|Mean_Temperature|Mean_Humidity|Mean_Wind_Speed|Mean_General_Diffuse_Flows|
|Mean_Diffuse_Flows|Mean_Power_Zone_1|Mean_Power_Zone_2|Mean_Power_Zone_3|
+-----+-----+-----+-----+-----+
```

+-----+-----+-----+-----+				
1	12.735	68.259	0.702	103.96
69.799		31052.984	19407.916	17736.352
2	12.657	66.491	1.114	125.471
92.331		30973.863	18774.586	17309.708
3	14.584	71.116	1.006	181.402
93.156		31162.869	18459.612	16945.463
4	16.415	75.408	0.223	157.722
83.495		31143.207	17600.307	18574.918
5	20.301	68.609	2.307	274.5
122.766		32379.46	19973.085	17604.283
6	22.133	68.761	1.561	277.435
103.228		34573.227	20649.035	20416.13
7	27.201	57.599	4.642	294.112
75.411		35805.53	24130.028	28175.034
8	25.74	66.023	4.533	227.179
67.106		36436.262	24657.025	24684.369
9	22.641	66.868	2.947	202.202
49.071		33415.103	20189.46	14928.416
10	20.476	71.524	2.784	115.815
46.629		32806.993	21457.89	13266.437
11	16.819	69.638	1.259	121.915
62.876		28993.342	23229.117	12867.073
12	13.283	69.237	0.255	100.023
34.305		28959.103	23618.565	11017.12
+-----+-----+-----+-----+				
+-----+-----+-----+-----+				

Pairing our means by month with their standard deviations is also a good idea. We would absolutely be interested in the variability of our data since that is the quality of our data that allows us to perform inference. Variability is quite stable across months for temperature, humidity, and wind speed. Humidity does have slightly more variability than other months during the months of seven and eight. `General_Diffuse_Flows` and `Diffuse_Flows` have more variability, the former in months five through nine, the latter months one through 6. All power zones have standard deviations that are higher between months six through nine relative to other months.

```
[10]: # Group the data by Month and find the standard deviations of the numeric
      ↪ variables
      # Compute standard deviation for numeric columns grouped by Month
      std_by_month = spark_df.groupBy("Month").agg(
          *[stddev(col_name).alias(f"StdDev_{col_name}") for col_name in
            ↪ ["Temperature", "Humidity", "Wind_Speed",
              ↪ "General_Diffuse_Flows", "Diffuse_Flows",
              ↪ "Power_Zone_1", "Power_Zone_2", "Power_Zone_3"]])
```

```
# Round all numeric standard deviation columns to 3 decimal places
rounded_std_by_month = std_by_month.select(
    "Month", # Keep Month column unchanged
    *[round(col(c), 3).alias(c) for c in std_by_month.columns if c != "Month"])

# Show the rounded table
rounded_std_by_month.show(truncate=False)
```

```
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+
-----+
|Month|StdDev_Temperature|StdDev_Humidity|StdDev_Wind_Speed|StdDev_General_Diffu
se_Flows|StdDev_Diffuse_Flows|StdDev_Power_Zone_1|StdDev_Power_Zone_2|StdDev_Pow
er_Zone_3|
+-----+-----+-----+-----+-----+
-----+
|1|3.241|12.156|1.612|166.165
|131.459|7402.323|4515.296|4436.997
|
|2|2.62|12.412|1.981|206.73
|169.156|6874.585|4390.391|4353.976
|
|3|3.759|13.918|1.901|260.149
|151.168|6782.137|4185.118|4256.766
|
|4|2.806|14.313|0.82|246.174
|123.912|6496.7|3835.629|4556.263
|
|5|3.3|16.436|2.408|331.999
|171.586|6809.333|4182.544|4353.394
|
|6|2.69|14.973|2.235|328.277
|143.498|7317.808|4465.664|5596.703
|
|7|3.857|18.85|1.111|331.734
|95.045|6966.074|4968.511|6913.958
|
|8|2.95|18.483|1.301|289.906
|90.663|7054.722|5163.443|6520.956
|
|9|2.878|15.993|2.293|270.173
|67.524|6471.367|4205.504|3425.856
|
|10|2.987|13.981|2.399|185.043
|69.421|6479.133|4612.671|3087.806
|
|11|3.683|12.832|2.075|184.692
```

123.142	5918.583	5452.892	3509.672
12 3.349	13.739	0.894	161.702
56.394	6176.465	5713.467	2840.343
+-----+-----+-----+-----+			
-----+-----+-----+-----+			
-----+			

```
[11]: # Before the conversion
spark_df.printSchema()

# Convert Hour column to DoubleType
spark_df = spark_df.withColumn("Hour", col("Hour").cast(DoubleType()))

# Verify the conversion
spark_df.printSchema() #Note type change for Month
```

```
root
|-- Temperature: double (nullable = true)
|-- Humidity: double (nullable = true)
|-- Wind_Speed: double (nullable = true)
|-- General_Diffuse_Flows: double (nullable = true)
|-- Diffuse_Flows: double (nullable = true)
|-- Power_Zone_1: double (nullable = true)
|-- Power_Zone_2: double (nullable = true)
|-- Power_Zone_3: double (nullable = true)
|-- Month: long (nullable = true)
|-- Hour: double (nullable = true)
```

Pipeline Setup and Elastic Net Fitting Now, we'll move on to building the machine learning pipeline. The purpose of this pipeline is to streamline the data transformation and modeling process. Data will enter one end, undergo a series of defined operations within the pipeline, and emerge at the other end in a format suitable for our analysis.

Our initial transformation addresses the distinction between day and night. Recognizing that weather patterns (including temperature, humidity, and wind speed) are naturally influenced by the presence of sunlight, we will create a binary feature to represent these two periods. The `Binarizer()` will generate a new column, `Night_Day`, assigning a value of 0 to hours below 6.5 and 1 to hours at or above this threshold.

Next, we'll apply One-Hot Encoding to our categorical features. This step is crucial for ensuring that our machine learning models do not incorrectly interpret these categories as continuous or ranked variables. By removing any implicit ordinal relationships, One-Hot Encoding ensures compatibility with MLlib, which expects numerical input features. For instance, in the `Month_OHE` example, 11 indicates the size of the resulting vector (number of categories minus one, due to zero-based indexing), and [3] signifies the position with a non-zero value (1.0), representing the active category for that observation.

Following feature encoding, we will employ Principal Component Analysis (PCA), as detailed in the linked resource. PCA is a dimensionality reduction technique that transforms a dataset with numerous correlated variables into a smaller set of uncorrelated principal components, effectively capturing the most important information.

With the data preprocessed through binarization, One-Hot Encoding, and PCA, we can now integrate our Elastic Net regression model into the pipeline. This stage involves first creating a predictor vector using `VectorAssembler()`, which combines the relevant feature columns into a single vector. This vector is then used to transform our Spark `DataFrame`. Since we are utilizing cross-validation across the entire dataset, we will not perform a separate train/test split at this stage. We will then instantiate the `LinearRegression()` model, often found in libraries like `scikit-learn` (though we are using the PySpark implementation here), to configure our Elastic Net model's parameters. Subsequently, we will proceed with the standard modeling procedure of fitting this model to our transformed `spark_df`.

Finally, to complete the pipeline setup, we will define a parameter grid for our Elastic Net model. We will leverage `RegressionEvaluator()` to configure the evaluation metric for our `CrossValidator()`. Cross-validation will partition our data into five folds, iteratively training and testing the model on different combinations of these folds to select the best performing model. We will use `RegressionEvaluator()` to extract the Root Mean Squared Error (RMSE) from the best model, followed by common methods to obtain the residuals and the corresponding best predictions.

```
[12]: # Define transformations
      # Change hours to night or day depending on hour
      # REF: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.
      ↪binarize.html
      Binarize = Binarizer(threshold=6.5, inputCol="Hour", outputCol="Night_Day")

      # Fix month to an index
      # REF: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.
      ↪OneHotEncoder.html
```

```

Encoder = OneHotEncoder(inputCol="Month", outputCol="Month_OHE")

# Performs Principal Component Analysis by first gathering feature columns,
↳ then funneling through PCA
# SRC_1 https://statisticsbyjim.com/basics/principal-component-analysis/
# SRC_2 https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.
↳ PCA.html
# SRC_3 https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.
↳ ml.feature.VectorAssembler.html
VectorAssembly = VectorAssembler(
    inputCols=[
        "Temperature", "Humidity", "Wind_Speed", "General_Diffuse_Flows",
↳ "Diffuse_Flows",
        "Night_Day", "Power_Zone_1", "Power_Zone_2", "Month_OHE"
    ],
    outputCol="final_features"
)
ComponentAnalysis = PCA(k=3, inputCol="final_features",
↳ outputCol="pca_features")

# Rename Power_Zone_3 to "label" for consistency with model input
spark_df = spark_df.withColumnRenamed("Power_Zone_3", "label")

# Define the Elastic Net model
# REF: https://scikit-learn.org/stable/modules/generated/sklearn.linear\_model.
↳ LinearRegression.html
elastic_net = LinearRegression(featuresCol="pca_features", labelCol="label")

# Create full transformation + model pipeline
# REF: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.
↳ Pipeline.html
pipeline = Pipeline(stages=[Binarize, Encoder, VectorAssembly,
↳ ComponentAnalysis, elastic_net])

# Define hyperparameter grid, parameters given in assignment.
# REF: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.
↳ ml.tuning.ParamGridBuilder.html
paramGrid = ParamGridBuilder() \
    .addGrid(elastic_net.regParam, [0, 0.05, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.
↳ 98, 0.99, 1]) \
    .addGrid(elastic_net.elasticNetParam, [0, 0.05, 0.1, 0.25, 0.5, 0.75, 0.9,
↳ 0.95, 0.98, 0.99, 1]) \
    .build()

# Evaluator for CrossValidation

```

```

# REF: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.
#       ↪ ml.evaluation.RegressionEvaluator.html
evaluator = RegressionEvaluator(labelCol="label", predictionCol="prediction",
#       ↪ metricName="r2")

# Cross-validation setup
# REF: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.
#       ↪ ml.tuning.CrossValidator.html
crossval = CrossValidator(estimator=pipeline, estimatorParamMaps=paramGrid,
#       ↪ evaluator=evaluator, numFolds=5)

# Fit model with cross-validation
cvModel = crossval.fit(spark_df) #No splitting the data into test/train, run
#       ↪ full dataset on CV.

# Transform test data using trained model
predictions = cvModel.transform(spark_df)

# add column "Residual"
predictions = predictions.withColumn("residual", predictions["label"] -
#       ↪ predictions["prediction"])

rmse = evaluator.evaluate(predictions)
print(f"Training Set RMSE: {rmse}")

```

Training Set RMSE: 0.5806586297035342

1.0.2 Modeling on Streaming

Having successfully modeled our static batch data, we will now turn our attention to processing streaming data. The initial step in this process involves enforcing a consistent data format for all incoming data to mitigate potential errors during ingestion. Subsequently, we will define the parameters of our data stream using `readStream`. This will specify crucial details such as the file format, the presence of headers, and the source location of the streaming data. In the transformation and aggregation phase, the identical pipeline that was applied to the static batch data will be utilized for the streaming data, with any necessary minor adjustments. Finally, we will join the streams containing both the predictions and the transformed features and execute a `loop.py` script. This script will simulate the continuous arrival of streaming data, allowing us to observe the real-time updates and uploads of our model's predictions.

Reading a Stream

```

[19]: # Define the schema for the incoming CSV files
# Ensure 'Hour' is DoubleType as requested
streaming_schema = StructType([
    StructField("Temperature", DoubleType(), True),
    StructField("Humidity", DoubleType(), True),

```



```

    StructField("Wind_Speed", DoubleType(), True),
    StructField("General_Diffuse_Flows", DoubleType(), True),
    StructField("Diffuse_Flows", DoubleType(), True),
    StructField("Power_Zone_1", DoubleType(), True),
    StructField("Power_Zone_2", DoubleType(), True),
    StructField("Power_Zone_3", DoubleType(), True),
    StructField("Month", LongType(), True),
    StructField("Hour", DoubleType(), True)
])

# Set the path to the directory where the streaming data will arrive
streaming_data_path = "Streaming_Data"

# Read the stream of CSV files
streaming_df = spark.readStream.format("csv") \
    .schema(streaming_schema) \
    .option("header", "true") \
    .option("pathGlobFilter", "*.csv") \
    .option("maxFilesPerTrigger", 1) \
    .load(streaming_data_path)

# Rename Power_Zone_3 to "label" in the streaming data
streaming_df = streaming_df.withColumnRenamed("Power_Zone_3", "label")

# As namesake suggests, prints the schema setup for data ingestion.
streaming_df.printSchema()

```

```

root
|-- Temperature: double (nullable = true)
|-- Humidity: double (nullable = true)
|-- Wind_Speed: double (nullable = true)
|-- General_Diffuse_Flows: double (nullable = true)
|-- Diffuse_Flows: double (nullable = true)
|-- Power_Zone_1: double (nullable = true)
|-- Power_Zone_2: double (nullable = true)
|-- label: double (nullable = true)
|-- Month: long (nullable = true)
|-- Hour: double (nullable = true)

```

Transform/Aggregation

```

[20]: # Apply the trained model to the incoming streaming data
predictions_stream = cvModel.transform(streaming_df) \
    .select("label", "prediction") \
    .withColumn("residual", col("label") - col("prediction"))

```

```

# Define a new pipeline for streaming transformations (excluding Elastic Net
↳model)
streaming_transformation_pipeline = Pipeline(stages=[Binarize, Encoder,
↳VectorAssembly, ComponentAnalysis])

# Fit this transformation pipeline on batch data before applying it to the
↳stream
fitted_streaming_transformation = streaming_transformation_pipeline.
↳fit(spark_df)

# Apply this fitted pipeline to incoming streaming data
transformed_stream = fitted_streaming_transformation.transform(streaming_df)

```

Writing Step

```

[ ]: # Join transformed streaming data with prediction results on 'label'
joined_stream = transformed_stream.join(predictions_stream, on="label",
↳how="inner")

# Write joined results to console
query = joined_stream.writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", "false") \
    .start()

query.awaitTermination()

```

Batch: 0

```

+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows|
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                                     |prediction
|residual      |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+
|25028.28452|26.71      |70.9     |4.924     |554.0      |297.8

```

```

|36767.04319 |24789.87342 |7      |15.0|1.0      |(12,[7],[1.0])|(20,[0,1,2,3,4,5,
6,7,15],[26.71,70.9,4.924,554.0,297.8,1.0,36767.04319,24789.87342,1.0])|[-
44344.6641546289,384.03977387881184,-353.01604248717393]|19853.04458535413
|5175.23993464587 |
|18717.09091|13.48      |87.1      |0.069      |78.2              |70.1
|37120.68891 |18810.1833 |4      |11.0|1.0      |(12,[4],[1.0])|(20,[0,1,2,3,4,5,
6,7,12],[13.48,87.1,0.069,78.2,70.1,1.0,37120.68891,18810.1833,1.0])
|[-41247.70829454484,5510.741097358986,175.0465181100291] |22628.09972286134
|-3911.0088128613424|
|14644.16752|24.18      |76.8      |0.228      |247.5              |144.9
|29711.15044 |20035.75884 |9      |9.0 |1.0      |(12,[9],[1.0])|(20,[0,1,2,3,4,5,
6,7,17],[24.18,76.8,0.228,247.5,144.9,1.0,29711.15044,20035.75884,1.0])|[-
35835.13368009702,306.5821606726544,-65.2700915077158]
|15648.404777407652|-1004.2372574076508|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+
-----+

-----
Batch: 1
-----
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                                |prediction
|residual      |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+
|15886.45161|13.25      |83.4      |0.078      |229.0              |217.1
|33653.10638 |20612.19512 |3      |11.0|1.0      |(12,[3],[1.0])|(20,[0,1,2,3,4,5,
6,7,11],[13.25,83.4,0.078,229.0,217.1,1.0,33653.10638,20612.19512,1.0])|[-
39410.71451102237,2063.2444523242307,-39.64488587194822]|18865.588652345716|-
2979.1370423457156|
|16573.74874|17.97      |55.97      |1.236      |510.3              |554.8
|35371.52542 |22074.16413 |2      |15.0|1.0      |(12,[2],[1.0])|(20,[0,1,2,3,4,5,
6,7,10],[17.97,55.97,1.236,510.3,554.8,1.0,35371.52542,22074.16413,1.0])|[-
41656.88749862006,1833.0878103941782,-397.2725788313259]|19211.3044688113

```

```
| -2637.5557288113014|
|18544.96425|27.47      |29.28      |4.927      |61.97      |55.71
|38599.64602 |22187.52599 |9          |18.0|1.0      |(12,[9],[1.0])|(20,[0,1,2,3,4,5,
6,7,17],[27.47,29.28,4.927,61.97,55.71,1.0,38599.64602,22187.52599,1.0])|[-
44378.80505020888,3563.979453603633,202.67275665966835]|23153.91523348666
|-4608.9509834866585|
```

Batch: 2

label	Temperature Humidity Wind_Speed General_Diffuse_Flows Diffuse_Flows
Power_Zone_1 Power_Zone_2 Month Hour Night_Day Month_OHE	final_features
pca_features	prediction
residual	

```
|16753.73494|15.64      |57.26      |0.077      |409.0      |44.9
|29571.64557 |17894.22492 |1          |15.0|1.0      |(12,[1],[1.0])|(20,[0,1,2,3,4,5,
6,7,9],[15.64,57.26,0.077,409.0,44.9,1.0,29571.64557,17894.22492,1.0])|[-
34508.54235156273,1993.2181522543474,-
189.92397386276835]|15754.982846547742|998.7520934522563  |
|11420.88836|11.56      |80.6       |0.082      |0.048      |0.126
|26476.04563 |22033.75268 |12        |0.0 |0.0      |(12,[],[])
|(20,[0,1,2,3,4,6,7],[11.56,80.6,0.082,0.048,0.126,26476.04563,22033.75268])
|[-34298.08238158797,-
3173.2719513125994,188.92720147257873]|13028.886667744258|-1607.9983077442575|
|15661.94478|12.21      |86.0       |0.078      |0.037      |0.156
|39622.81369 |33746.54802 |12        |20.0|1.0      |(12,[],[]) |(20,[0,1,2,3,4,5,
6,7],[12.21,86.0,0.078,0.037,0.156,1.0,39622.81369,33746.54802])
|[-51765.919973473276,-5385.225240491029,281.5949730085624]
|21545.627724378617|-5883.682944378617 |
```

```

-----+-----
-----+-----
-----+

```

Batch: 3

```

-----+-----
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                                |prediction
|residual      |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|28547.21003|29.88      |65.19   |4.904     |334.9           |212.5
|40281.64262|29491.86906|8       |16.0|1.0        |(12,[8],[1.0])|(20,[0,1,2,3,4,5
,6,7,16],[29.88,65.19,4.904,334.9,212.5,1.0,40281.64262,29491.86906,1.0])|[-
49902.364567581186,-1503.1783825539612,-
94.7550648345892]|22267.096959482114|6280.113070517884  |
|26332.2884 |26.79      |77.2    |4.928     |223.4           |168.4
|35058.64595 |22565.5755 |8       |17.0|1.0        |(12,[8],[1.0])|(20,[0,1,2,3,4,5
,6,7,16],[26.79,77.2,4.928,223.4,168.4,1.0,35058.64595,22565.5755,1.0])|[-
41675.10743599852,1248.6455571531835,-
10.512650374882497]|19657.273057368417|6675.015342631585  |
|12039.3921 |21.2       |71.8    |4.924     |88.5            |73.3
|32114.83589 |25760.16598|10      |17.0|1.0        |(12,[10],[1.0])|(20,[0,1,2,3,4,5
,6,7,18],[21.2,71.8,4.924,88.5,73.3,1.0,32114.83589,25760.16598,1.0])
|[-41056.3385297614,-3052.056469159776,122.77856238406984]|16752.7107888111
|-4713.3186888110995|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
-----+

```

Batch: 4

```

-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                |prediction
|residual      |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|12780.54711|26.53      |45.45   |4.918    |240.2                |86.4
|34767.96499 |20502.07469 |10      |16.0|1.0      |(12,[10],[1.0])|(20,[0,1,2,3,4,5
,6,7,18],[26.53,45.45,4.918,240.2,86.4,1.0,34767.96499,20502.07469,1.0])|[-
40267.27106105326,2784.8424549058104,-3.993734118146794]|19897.717556906377|[-
7117.170446906377|
|9220.668693|17.91      |67.54   |0.086    |26.67                |23.18
|26279.21225 |21506.639   |10      |8.0 |1.0      |(12,[10],[1.0])|(20,[0,1,2,3,4,5
,6,7,18],[17.91,67.54,0.086,26.67,23.18,1.0,26279.21225,21506.639,1.0])
|[-33837.5980323684,-2850.039040552889,155.08140832584763]
|12912.661725958676|-3691.993032958677|
|12092.53012|8.49        |85.4    |0.088    |0.037                |0.122
|23009.62025 |13812.76596 |1       |7.0 |1.0      |(12,[1],[1.0])|(20,[0,1,2,3,4,5
,6,7,9],[8.49,85.4,0.088,0.037,0.122,1.0,23009.62025,13812.76596,1.0])|[-
26786.43042257827,1640.7923082812545,167.29377953972707]|11957.230295497453|135.
29982450254647|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+

```

Batch: 5

```

-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features

```



```
|46930.76586 |29165.9751 |10 |19.0|1.0 |(12,[10],[1.0])|(20,[0,1,2,3,4,5
,6,7,18],[20.4,88.8,0.077,0.055,0.119,1.0,46930.76586,29165.9751,1.0])
|[-55196.38641123979,2527.8266792066274,338.68477442478957] |28815.74215595278
|-9265.59017595278 |
|22825.80905|11.6 |89.6 |0.085 |0.055 |0.126
|37061.69492 |23168.38906 |2 |22.0|1.0 |(12,[2],[1.0])|(20,[0,1,2,3,4,5
,6,7,10],[11.6,89.6,0.085,0.055,0.126,1.0,37061.69492,23168.38906,1.0])
|[-43665.96852705279,1884.3757495037557,267.87840736764707] |21783.66684266842
|1042.1422073315807|
|25934.47236|13.45 |64.75 |0.082 |45.91 |45.65
|41924.74576 |27074.77204 |2 |18.0|1.0 |(12,[2],[1.0])|(20,[0,1,2,3,4,5
,6,7,10],[13.45,64.75,0.082,45.91,45.65,1.0,41924.74576,27074.77204,1.0])|[-
49886.419622216774,1417.8134899873462,244.39149830511275] |24908.678353923446|102
5.794006076554 |
```

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
-----+
```

Batch: 7

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
-----+
|label |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE |final_features
|pca_features |prediction
|residual |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
```

```
|11490.82067|18.59 |81.7 |4.915 |0.033 |0.148
|24804.55142 |15613.69295 |10 |3.0 |0.0 |(12,[10],[1.0])|(20,[0,1,2,3,4,6
,7,18],[18.59,81.7,4.915,0.033,0.148,24804.55142,15613.69295,1.0])
|[-29285.56057689333,1172.470390065975,179.64829129585289]
|13073.476812373168|-1582.6561423731691|
|13129.06883|21.55 |80.3 |4.926 |473.7 |433.1
|28579.67213 |17435.29412 |5 |9.0 |1.0 |(12,[5],[1.0])|(20,[0,1,2,3,4,5
,6,7,13],[21.55,80.3,4.926,473.7,433.1,1.0,28579.67213,17435.29412,1.0])|[-
33431.840571020744,1811.2922717904526,-374.0523274981633] |14640.603438573971|-
```



```

1511.5346085739711|
|15965.09091|14.81      |87.4    |0.074    |0.033      |0.074
|24032.03445 |12999.59267 |4      |1.0 |0.0      |(12,[4],[1.0]) |(20,[0,1,2,3,4,6
,7,12],[14.81,87.4,0.074,0.033,0.074,24032.03445,12999.59267,1.0])
|[-27168.801891708477,2889.915744265582,175.3854777517485]
|13013.790667881769|2951.300242118232 |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
-----+

```

Batch: 8

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                                |prediction
|residual      |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
+
|16758.99497|17.15      |35.45    |0.086    |570.5      |102.4
|33339.66102 |19455.31915 |2      |15.0|1.0      |(12,[2],[1.0]) |(20,[0,1,2,3,4,5
,6,7,10],[17.15,35.45,0.086,570.5,102.4,1.0,33339.66102,19455.31915,1.0])|[-
38499.26748955725,2840.428325274627,-334.26119367554304]
|18242.602956978328|-1483.607986978328|
|11356.59574|20.81      |64.78    |4.913    |441.5      |42.89
|30091.90372 |24942.32365 |10     |15.0|1.0      |(12,[10],[1.0]) |(20,[0,1,2,3,4,5
,6,7,18],[20.81,64.78,4.913,441.5,42.89,1.0,30091.90372,24942.32365,1.0])|[-
38927.87983895567,-3521.8428206934154,-219.89356665970698]|14522.301213864454|-
3165.705473864453|
|14503.38462|23.95      |43.98    |0.085    |791.0      |183.3
|27630.19868 |14029.52183 |6      |10.0|1.0      |(12,[6],[1.0]) |(20,[0,1,2,3,4,5
,6,7,14],[23.95,43.98,0.085,791.0,183.3,1.0,27630.19868,14029.52183,1.0])|[-
30722.612179649317,4081.6771841414115,-608.6363548590787]
|14125.024237078074|378.36038292192643|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+

```

```

-----+-----
-----+-----
-----+

Batch: 9
-----
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                |prediction
|residual                    |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
|18176.0    |20.02      |51.92   |0.083    |404.0      |383.4
|32972.74489|18593.89002|4       |17.0|1.0      |(12,[4],[1.0])|(20,[0,1,2,3,4,5
,6,7,12],[20.02,51.92,0.083,404.0,383.4,1.0,32972.74489,18593.89002,1.0])|[-
37708.4517373361,3342.9153077825163,-261.0733519697242]
|18287.287848318774|-111.2878483187742|
|11432.46201|24.22      |66.07   |4.923    |607.1      |87.2
|34931.81619|23239.41909|10      |13.0|1.0      |(12,[10],[1.0])|(20,[0,1,2,3,4,5
,6,7,18],[24.22,66.07,4.923,607.1,87.2,1.0,34931.81619,23239.41909,1.0])
|[-41954.213649755795,622.6968825176697,-353.9749727834278]
|18670.137966941693|-7237.675956941694|
|16414.83871|13.62      |80.2    |0.076    |435.5      |354.4
|34474.21277|20787.80488|3       |11.0|1.0      |(12,[3],[1.0])|(20,[0,1,2,3,4,5
,6,7,11],[13.62,80.2,0.076,435.5,354.4,1.0,34474.21277,20787.80488,1.0])
|[-40188.30769231522,2384.5647219779244,-271.62155210241394]|19020.32627832053
|-2605.487568320528|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
-----+

```

Batch: 10

```

-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+
|label      |Temperature|Humidity|Wind_Speed|General_Diffuse_Flows|Diffuse_Flows
|Power_Zone_1|Power_Zone_2|Month|Hour|Night_Day|Month_OHE      |final_features
|pca_features                |prediction
|residual                    |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
|12423.87692|18.94      |60.78   |0.082    |186.5                |150.2
|26180.66225 |16832.43243 |6       |8.0 |1.0      |(12,[6],[1.0])|(20,[0,1,2,3,4,5,
6,7,14],[18.94,60.78,0.082,186.5,150.2,1.0,26180.66225,16832.43243,1.0])|[-
31111.11134246049,948.1014051109374,-33.81041657408397]|13493.85507799047
|-1069.9781579904702|
|26427.20648|18.25      |76.8    |4.92     |0.037                |0.104
|46671.7377  |28246.43963 |5       |21.0|1.0      |(12,[5],[1.0])|(20,[0,1,2,3,4,5,
6,7,13],[18.25,76.8,4.92,0.037,0.104,1.0,46671.7377,28246.43963,1.0])
|[-54462.28192123974,3139.147839770961,336.9580132911838]|28804.88786990557
|-2377.6813899055705|
|10050.8502 |19.36      |69.81   |0.074    |3.072                |2.624
|21365.5082  |13292.87926 |5       |6.0 |0.0      |(12,[5],[1.0])|(20,[0,1,2,3,4,6,
7,13],[19.36,69.81,0.074,3.072,2.624,21365.5082,13292.87926,1.0])|[-
25136.92190557044,1138.5577744159596,151.1348899341701]|10667.433403654219|-
616.5832036542179 |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
-----+
-----+

```

[Stage 11237:=>

(7 + 128) / 200]

The stream stops when CTRL+C is used to force stoppage or when 50 iterations of the loop run—whichever is first. At 10 second intervals a complete run would be just under 9 minutes. In the end we see our batch being processed one-by-one as they arrive to the environment, joining the prediction_stream containing the cvModel transformation with the Pipeline transformation.